

- Dashboard: revenue, orders, customers, pending orders, low stock and top products.
- Products: add/edit/delete, images, pricing, SKU, stock, availability and featured status.
- Categories: categories, subcategories, ordering and status.
- Orders: view, confirm, prepare, ready, delivery, pickup, cancellation and refund workflow.
- Customers: profiles, addresses, order history, spending and repeat-customer data.
- Custom Cakes: review requests, quotations, reference images, status and notes.
- Inventory: stock, reserved stock, adjustments, waste, returns and transfers.
- Offers/Coupons: percentage/fixed discounts, limits and expiry.
- Reviews: moderation and customer feedback.
- Branches: locations, opening hours and branch-level stock/sales.
- Staff: roles, permissions and activity.
- Reports/Analytics: sales, channels, products, customers, profit and operational metrics.
- Settings: bakery information, delivery, tax, payment and notification settings.
- Audit Logs: track important administrative changes.

7. Phase 2 POS / Billing

The POS should not use a separate database. It should create sales through the same backend and orders model. The source field identifies whether an order came from WEBSITE, POS, PHONE or WHATSAPP.

POS
↓
Search / Barcode Scan
↓
Add Items
↓
Customer (optional)
↓
Discount
↓
Payment
↓
Complete Sale
↓
Receipt
↓
Inventory Transaction
↓
Analytics

- Barcode scanner support for packaged products.
- Cash, card and configured payment methods.
- 58mm/80mm thermal receipt printing.
- Returns and refunds.
- Cashier/manager permissions.
- Daily cash closing and reconciliation.
- Branch-specific POS.
- Offline queue/sync can be added later if the bakery requires it.

8. Database Design

Core relational tables:

users
roles
permissions
role_permissions
branches
addresses
categories
products
product_images
carts
cart_items
orders
order_items
payments
order_status_history
custom_cake_requests
reviews
wishlists
wishlist_items
coupon_usages
inventory_transactions
notifications
expense
returns
audit_logs
settings

Important design rule: use one central PostgreSQL database. Website orders and POS sales should both enter the same orders/order_items structure. Inventory changes should be recorded as transactions rather than silently overwriting stock.

9. Key Database Fields

Table	Important fields
users	id, name, email, phone, password_hash, role, status, created_at
products	id, sku, barcode, category_id, name, description, price, sale_price, cost_price, stock settings, status
orders	id, order_number, customer_id, branch_id, order_source, order_type, status, payment_status, totals, address, schedule
order_items	id, order_id, product_id, product_name, quantity, unit_price, discount, total
payments	id, order_id, method, transaction_reference, amount, status, paid_at
inventory	id, product_id, branch_id, quantity, reserved_quantity, minimum_quantity
inventory_transactions	id, product_id, branch_id, type, quantity, reference_type, reference_id, created_by
custom_cake_requests	id, customer_id, type, flavour, size, cream, theme, message, image, date, quote, status
branches	id, name, code, address, city, phone, opening_time, closing_time, status
expenses	id, branch_id, category, description, amount, expense_date, created_by
audit_logs	id, user_id, action, entity_type, entity_id, old_data, new_data, created_at

10. Order & Inventory Rules

- Order status and payment status must be separate.
- Stock should support reserved_quantity to prevent overselling.
- A successful order reserves/deducts stock according to the configured business rule.
- Cancellation releases reserved stock where appropriate.
- Returns create a RETURN inventory transaction.
- POS and website use the same product and inventory records.
- Every stock adjustment should have a transaction record and responsible user.
- Historical order_items should preserve product name and price snapshots.

11. API Structure

/api/v1/auth
/api/v1/products
/api/v1/categories
/api/v1/cart
/api/v1/orders
/api/v1/payments
/api/v1/custom-cakes
/api/v1/reviews
/api/v1/coupons
/api/v1/customers
/api/v1/inventory
/api/v1/branches
/api/v1/staff
/api/v1/reports
/api/v1/analytics
/api/v1/notifications
/api/v1/pos

Example REST operations:

```
GET /products  
GET /products/{id}  
POST /products  
PUT /products/{id}  
DELETE /products/{id}  
POST /orders  
GET /orders  
GET /orders/{id}  
PATCH /orders/{id}/status  
POST /pos/sales  
POST /pos/returns
```

12. Recommended Flutter Project Structure

```
lib/  
  core/  
  config/  
  theme/  
  routing/  
  network/  
  storage/  
  utils/  
  features/  
  auth/  
  home/  
  catalog/  
  product/  
  cart/  
  checkout/  
  orders/  
  custom_cake/  
  reviews/  
  customer/  
  admin/  
  pos/  
  shared/  
  widgets/  
  models/  
  services/  
  main.dart
```

13. Backend Structure

```
backend/  
  app/  
  main.py  
  api/v1/  
  models/  
  schemas/  
  services/  
  repositories/  
  core/  
  utils/  
  migrations/  
  tests/  
  requirements.txt  
  .env
```

14. Roles & Permissions

Role	Typical permissions
Customer	Browse, cart, checkout, orders, reviews, custom cakes
Cashier	POS sales, receipts, customer lookup, permitted discounts
Manager	POS, refunds, inventory, operational reports, discounts
Admin	Products, orders, customers, offers, inventory, staff, reports
Owner	Full access, financial/analytics/settings, staff control

15. Security Requirements

- Hash passwords with Argon2id or bcrypt.
- Use secure authentication/session handling and short-lived access credentials where applicable.
- Role-based authorization on every protected backend action.
- Validate all API input with Pydantic.
- Use parameterized ORM/database queries.
- Restrict CORS to trusted origins.
- Rate-limit login and sensitive endpoints.
- Validate file type, size and storage path for image uploads.
- Never commit .env or API/payment secrets to GitHub.
- Use HTTPS in production.
- Back up PostgreSQL regularly.
- Maintain audit logs for sensitive administrative changes.

16. Notifications & Integrations

- Email: order confirmations, receipts and password recovery.
- WhatsApp: optional order/status messaging if a suitable provider is connected.
- Payment gateway: integrate according to the bakery's selected provider and country requirements.
- Maps: branch locations and delivery addresses.
- Push notifications: Android/iOS when mobile apps are deployed.
- PDF invoice/receipt generation.

17. Analytics

- Revenue by day/week/month/custom range.
- Orders and average order value.
- Website vs POS vs WhatsApp vs phone sales.
- Top products and categories.
- New vs returning customers.
- Cancelled orders and refunds.
- Low-stock products.
- Gross profit using cost_price.

- Branch-level performance.
- Cashier/shift reports.

18. Deployment Architecture

GitHub
 ■ CI/CD Pipeline
 ■
 ↓ ↓ Flutter
 Web Build Flutter Apps
 ↓ Android/iOS/Desktop
 Web Hosting
 ■
 ↓ FastAPI Server
 ■
 ↓ PostgreSQL Object Storage
 ■ Redis (optional)

The exact hosting provider can be chosen later based on budget, expected traffic and whether the bakery needs local POS hardware. The architecture remains provider-neutral.

19. Development Phases

Phase	Scope
1	Flutter customer website/app foundation, Home, Catalog, Product Details, Search, Gallery, Contact
2	Accounts, Cart, Checkout, Delivery/Pickup, Orders, Reviews, Coupons
3	FastAPI backend hardening, Admin Dashboard, Products, Orders, Customers, Custom Cakes, Inventory
4	Payments, Notifications, Analytics, Reports, Security, Deployment
5	POS, Billing, Barcode, Thermal Printing, Returns, Daily Closing
6	Expenses, Profit Analytics, Multi-Branch, Advanced Inventory, Audit Logs
7	Optional offline POS, advanced recommendations, loyalty and marketing automation

20. Client-Facing Package

Phase 1: Online Bakery Platform

- Responsive web experience
- Product/menu management
- Online ordering
- Custom cake requests
- Delivery/pickup
- Customer accounts
- Admin dashboard

Phase 2: Bakery POS

- Counter billing
- Barcode scanning
- Thermal receipt
- Cashier accounts
- Returns/refunds
- Daily closing
- Inventory synchronization

Phase 3: Business Intelligence

- Advanced inventory

- Expenses
- Profit analytics
- Multi-branch
- Advanced reports
- Customer/loyalty features

21. Final Recommendation

For this particular requirement, use Flutter as the shared frontend because the target includes web, Android, iOS, macOS and Linux. Keep the backend independent with FastAPI and PostgreSQL. This gives you one UI codebase while preserving a clean server architecture. Do not put business rules only inside the Flutter app. Orders, payments, inventory, permissions and pricing must be enforced by the backend.

Target: one Flutter project + one FastAPI backend + one PostgreSQL database + shared storage/services. Add POS later without replacing the website or creating a second database.

Document prepared: 25 September 2026